

SANJAY SENTHILKUMAR

Software Engineering: Building production RAG pipelines, LLM integrations, and cloud-backed APIs (Python, FastAPI, React).

New Brunswick, NJ | 856-500-9878 | Sansenus123@gmail.com | [linkedin.com/in/sanjay-senthilkumar-804b8b27b/](https://www.linkedin.com/in/sanjay-senthilkumar-804b8b27b/) | github.com/Sanj84-co

EDUCATION

Rutgers University

Bachelor of Science in Computer Science GPA: 3.61/4.0

Minor: Quantitative Economics

Relevant Coursework: Data Structures, Design and Analysis of Algorithms, Computer Architecture, Discrete Structures, Introduction to Data Management

New Brunswick, NJ

May 2028 (Expected)

TECHNICAL SKILLS

Languages: Python, Java, JavaScript, TypeScript, C, SQL, HTML5, CSS **Certifications:** AWS Certified AI Practitioner

Frameworks & Libraries: React, React Native, Node.js, Express, FastAPI, LangChain, Pandas, NumPy, Scikit-Learn, Matplotlib

Data & Infrastructure: PostgreSQL, Redis, Neo4j, Pinecone, FAISS, DuckDB, Supabase, Apache Spark, Databricks, Delta Lake, Docker, CI/CD (GitHub Actions), REST APIs, Git, GitHub, AWS, Azure, GCP

AI/LLM Tooling: RAG pipelines, vector embeddings, prompt engineering, model fine-tuning, knowledge distillation, quantization, model evaluation (RAGAS), PyTorch, Claude Code, GitHub Copilot, Cursor

EXPERIENCE

VIRTUSA | Python, LangChain, FastAPI, Pinecone, DuckDB, Docling

Jun 2026 - Present

AI Engineer Intern

Remote

- Built a self-healing RAG observability system that monitors chatbot answer quality using RAGAS metrics and detects data and concept drift against DuckDB baselines.
- Automated corrective actions (re-ingestion, re-ranking) with escalation to engineers after three failed retries, removing manual intervention from the quality loop.
- Designed a 12-table schema with three-role RBAC (Admin, Junior, Client) enforcing strict data isolation, plus a 53-pair golden dataset across 11 documents for pre-production evaluation.

BETTRLABS | Python, LangChain, GraphRAG, Neo4j

Mar 2026 - Present

Software Engineer Intern

Remote

- Engineered a Neo4j knowledge graph modeling brand identity across 10+ entity types and 6+ relationship types.
- Built a graph-grounded RAG pipeline using 5+ vector embeddings per node for similarity retrieval, reducing ungrounded LLM outputs by anchoring responses in structured graph context.

RUMAD

Sep 2025 - Dec 2025

Backend Engineering Fellow (Software Engineer Intern)

New Brunswick, NJ

- Developed a RESTful backend with JavaScript, Express, and Supabase, owning the data layer and authentication for RUMAD's mentorship web application.

TECHNICAL PROJECTS

Exoplanet Explorer | Python, FastAPI, PostgreSQL, React, PySpark, FAISS | [GitHub](#) | [Live](#) Independent Project

- Built an ETL pipeline ingesting 900+ exoplanets from NASA's Exoplanet Archive into PostgreSQL with a Redis cache-aside layer, cutting average response time ~30% (23.8ms to 16.6ms); added a parallel PySpark/Databricks pipeline (groupBy, join) writing to a Delta Lake table, served through a React frontend.
- Implemented a RAG pipeline from scratch: parsed arXiv PDFs, chunked and embedded with sentence-transformers, indexed in FAISS with URL-based dedup; built an eval harness scoring retrieval rank (avg. 1.07 across 25 papers, 15-question benchmark) and hallucination resistance on out-of-domain queries.

Vision-RAG QA | ColQwen2, Qdrant, Gemini, LangGraph, FastAPI | [GitHub](#) | [Live](#)

Independent Project

- Built a vision-based RAG pipeline answering questions directly from PDF page images (ColPali/ColQwen2), avoiding OCR; diagnosed a retrieval failure on a dense SEC filing and built a two-stage reranking fix raising top-3 accuracy from 17% to 43% (+26pp) across a 30-question benchmark.
- Load-tested the service, found and fixed two concurrency bugs (a local-storage race and a model-loading race) with thread locking, verified under real concurrent load in production.
- Deployed via FastAPI on Hugging Face Spaces with CI/CD, API key auth, and structured logging; found and fixed a silent-hallucination bug where empty retrieval results produced ungrounded answers.

StockSee | Python, FastAPI, PostgreSQL, Redis, JWT, APScheduler, pytest | [GitHub](#) | [Live](#) Independent Project

- Root-caused and fixed a serialized-delivery race in the alert dispatcher, raising same-tick alert accuracy from 1/5 to 5/5.
- Built a JWT-authenticated stock API (13 endpoints) with a 95% cache hit rate, ~30x latency reduction (47ms to 1.6ms), and 1,534 req/sec throughput at p95 22.3ms (40% pytest coverage).